

HDOC Day-10 Activity

Question 1:

Write a C program to input time in seconds and convert it into days, hours, minutes, and seconds format.

Step 1: Understanding the Problem

- Input: Total time in seconds.
- Output: Equivalent time in days, hours, minutes, and seconds.
- Formula: days = total / 86400; remaining = total % 86400; hours = remaining / 3600; remaining = remaining % 3600; minutes = remaining / 60; seconds = remaining % 60.

Step 2: Standard Test Case Table

Test Case	Input	Expected Output	Actual Output	Result
1	90061 s	1 d 1 h 1 m 1 s	1 d 1 h 1 m 1 s	Pass
2	3661 s	0 d 1 h 1 m 1 s	0 d 1 h 1 m 1 s	Pass
3	172800 s	2 d 0 h 0 m 0 s	2 d 0 h 0 m 0 s	Pass

Step 3: Algorithm

1. Start.
2. Declare total_seconds, days, hours, minutes, seconds and remaining as integer variables.
3. Read total_seconds from the user.
4. Calculate days = total_seconds / 86400.
5. Calculate remaining = total_seconds % 86400.
6. Calculate hours = remaining / 3600.
7. Update remaining = remaining % 3600.
8. Calculate minutes = remaining / 60.
9. Calculate seconds = remaining % 60.
10. Display days, hours, minutes and seconds.
11. Stop.

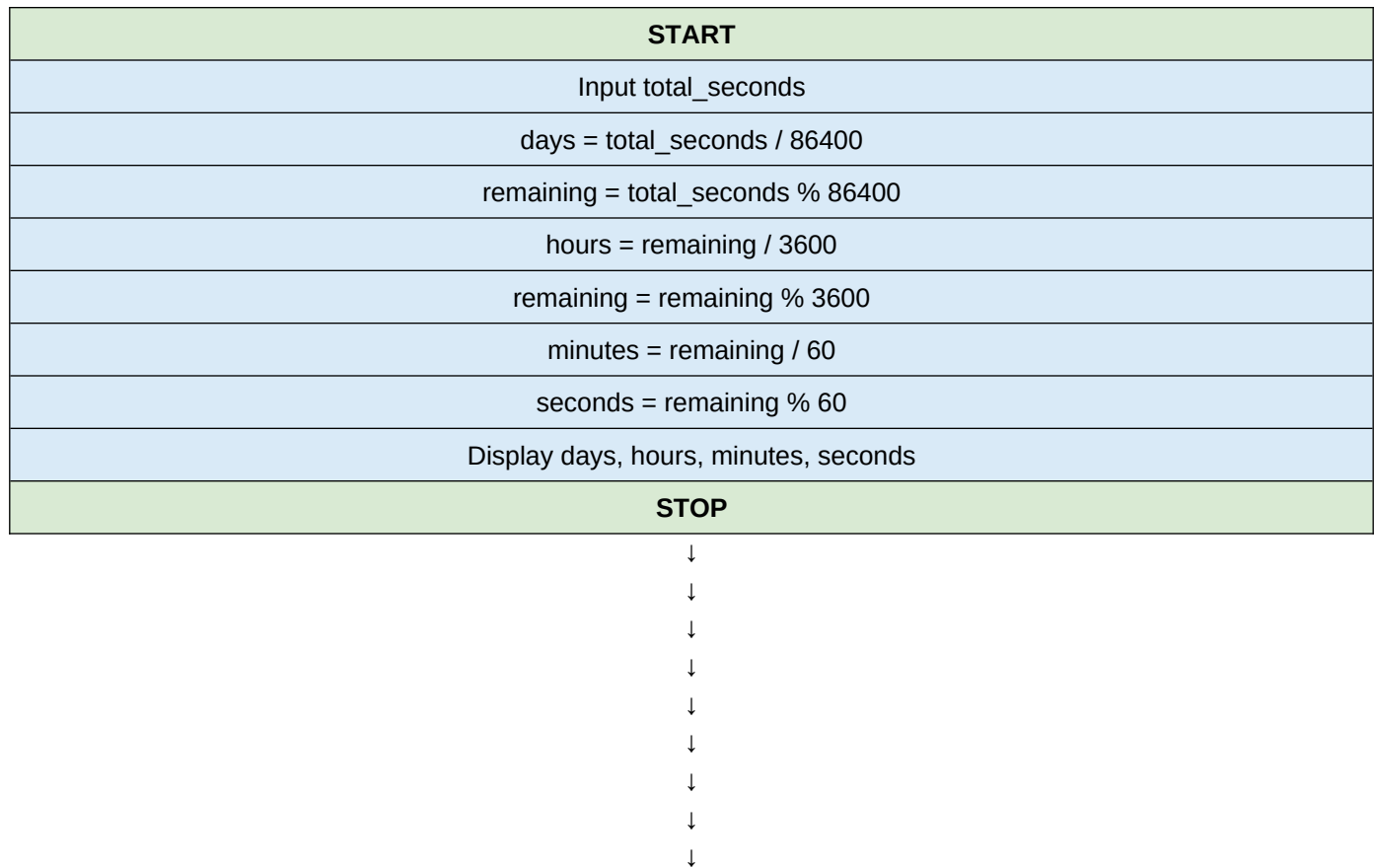
Evaluation: The algorithm successively removes complete days, hours and minutes from the total number of seconds. The remainder gives the seconds, so all prepared test cases are converted correctly.

Step 4: Pseudocode

```
BEGIN
INPUT total_seconds
days = total_seconds / 86400
remaining = total_seconds % 86400
hours = remaining / 3600
remaining = remaining % 3600
minutes = remaining / 60
seconds = remaining % 60
```

OUTPUT days, hours, minutes, seconds
END

Step 5: Flowchart



Flowchart Evaluation: The flowchart follows the same sequence as the algorithm and produces the expected output for each prepared test case.

Step 6: C Programming

```
#include <stdio.h>

int main()
{
    long long total_seconds, remaining;
    long long days, hours, minutes, seconds;

    printf("Enter time in seconds: ");
    scanf("%lld", &total_seconds);

    days = total_seconds / 86400;
    remaining = total_seconds % 86400;
    hours = remaining / 3600;
    remaining = remaining % 3600;
    minutes = remaining / 60;
    seconds = remaining % 60;

    printf("%lld days, %lld hours, %lld minutes, %lld seconds\n",
           days, hours, minutes, seconds);
    return 0;
}
```

Step 7: Kali Linux Compilation and Execution

```
(kali~kali)-[~]
$ nano time.c
(kali~kali)-[~]
$ gcc time.c -o time
(kali~kali)-[~]
$ ./time
Enter time in seconds: 90061
1 days, 1 hours, 1 minutes, 1 seconds
(kali~kali)-[~]
$ ./time
Enter time in seconds: 3661
0 days, 1 hours, 1 minutes, 1 seconds
(kali~kali)-[~]
$ ./time
Enter time in seconds: 172800
2 days, 0 hours, 0 minutes, 0 seconds
```

Question 2:

Write a C program to input a year and check whether it is a leap year or not.

Step 1: Understanding the Problem

- Input: A year as an integer.
- Output: Whether the entered year is a leap year or not.
- Condition: A year is a leap year if it is divisible by 400, or if it is divisible by 4 but not divisible by 100.

Step 2: Standard Test Case Table

Test Case	Input	Expected Output	Actual Output	Result
1	2024	Leap Year	Leap Year	Pass
2	1900	Not a Leap Year	Not a Leap Year	Pass
3	2000	Leap Year	Leap Year	Pass

Step 3: Algorithm

1. Start.
2. Declare year as an integer variable.
3. Read year from the user.
4. If year is divisible by 400, display "Leap Year".
5. Else if year is divisible by 100, display "Not a Leap Year".
6. Else if year is divisible by 4, display "Leap Year".
7. Otherwise display "Not a Leap Year".
8. Stop.

Evaluation: The algorithm correctly handles century years by checking divisibility by 400 before the general divisibility-by-4 rule. Therefore 1900 is rejected while 2000 is accepted.

Step 4: Pseudocode

```
BEGIN
INPUT year
IF year % 400 == 0 THEN
    OUTPUT "Leap Year"
ELSE IF year % 100 == 0 THEN
    OUTPUT "Not a Leap Year"
ELSE IF year % 4 == 0 THEN
    OUTPUT "Leap Year"
ELSE
    OUTPUT "Not a Leap Year"
END IF
END
```

Step 5: Flowchart

START
Input year
year % 400 == 0 ?
YES: Display Leap Year
NO: year % 100 == 0 ?
YES: Display Not a Leap Year
NO: year % 4 == 0 ?
YES: Display Leap Year
NO: Display Not a Leap Year
STOP



Flowchart Evaluation: The decision sequence correctly distinguishes ordinary leap years, century non-leap years, and century leap years.

Step 6: C Programming

```
#include <stdio.h>

int main()
{
    int year;
    printf("Enter year: ");
    scanf("%d", &year);

    if (year % 400 == 0)
        printf("Leap Year\n");
    else if (year % 100 == 0)
        printf("Not a Leap Year\n");
    else if (year % 4 == 0)
        printf("Leap Year\n");
    else
        printf("Not a Leap Year\n");

    return 0;
}
```

Step 7: Kali Linux Compilation and Execution

```
(kali~kali)-[~]
$ nano leapyear.c
(kali~kali)-[~]
$ gcc leapyear.c -o leapyear
(kali~kali)-[~]
$ ./leapyear
Enter year: 2024
Leap Year
(kali~kali)-[~]
$ ./leapyear
Enter year: 1900
Not a Leap Year
(kali~kali)-[~]
$ ./leapyear
Enter year: 2000
Leap Year
```